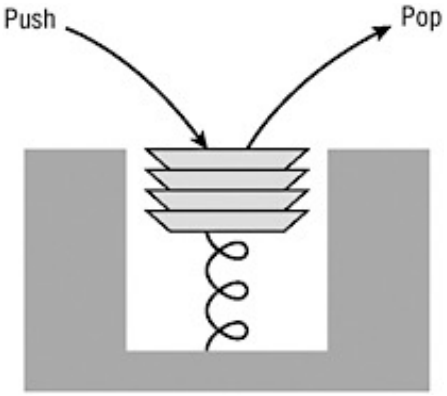
Datastrukturen Stack

Ännu ett exempel på hur man skriver en klass

Datastrukturen Stack



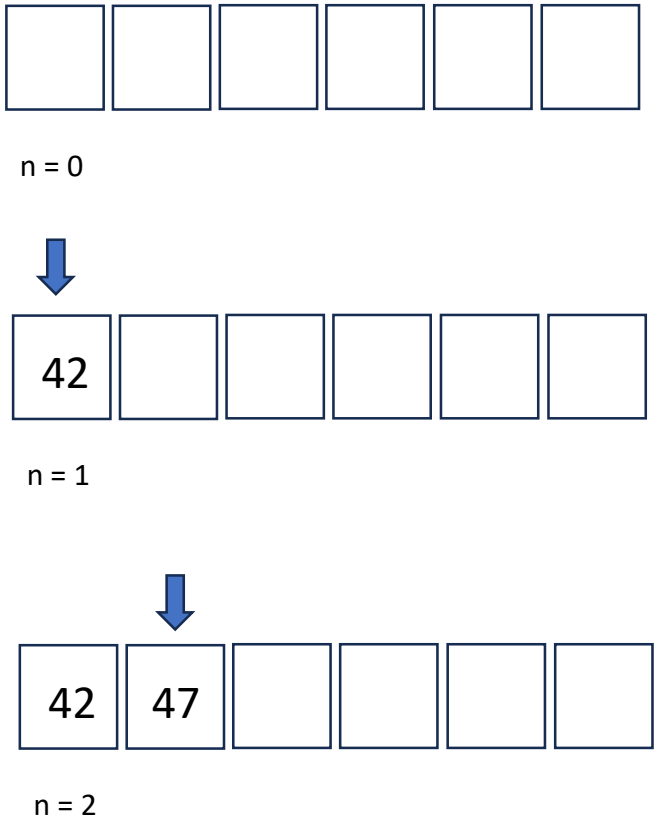
- En stack kan lagra en uppsättning objekt, med kontraktet att …
  - nya objekt placeras högst upp, på toppen,
  - objekt tas bort från toppen - tänk en hög med tallrikar
  - Det ska inte gå att modifiera stacken på annat sätt
- Klassen ska alltså endast ha två metoder för att modifiera stacken (mutatorer)
  - push – nytt objekt placeras på toppen
  - pop – tar bort, och returnerar objektet som ligger på toppen
- Andra metoder
  - size – antalet objekt i stacken
  - isEmpty
  - peek – returnera en referens till objektet på toppen utan att ta bort det

Datastrukturen Stack

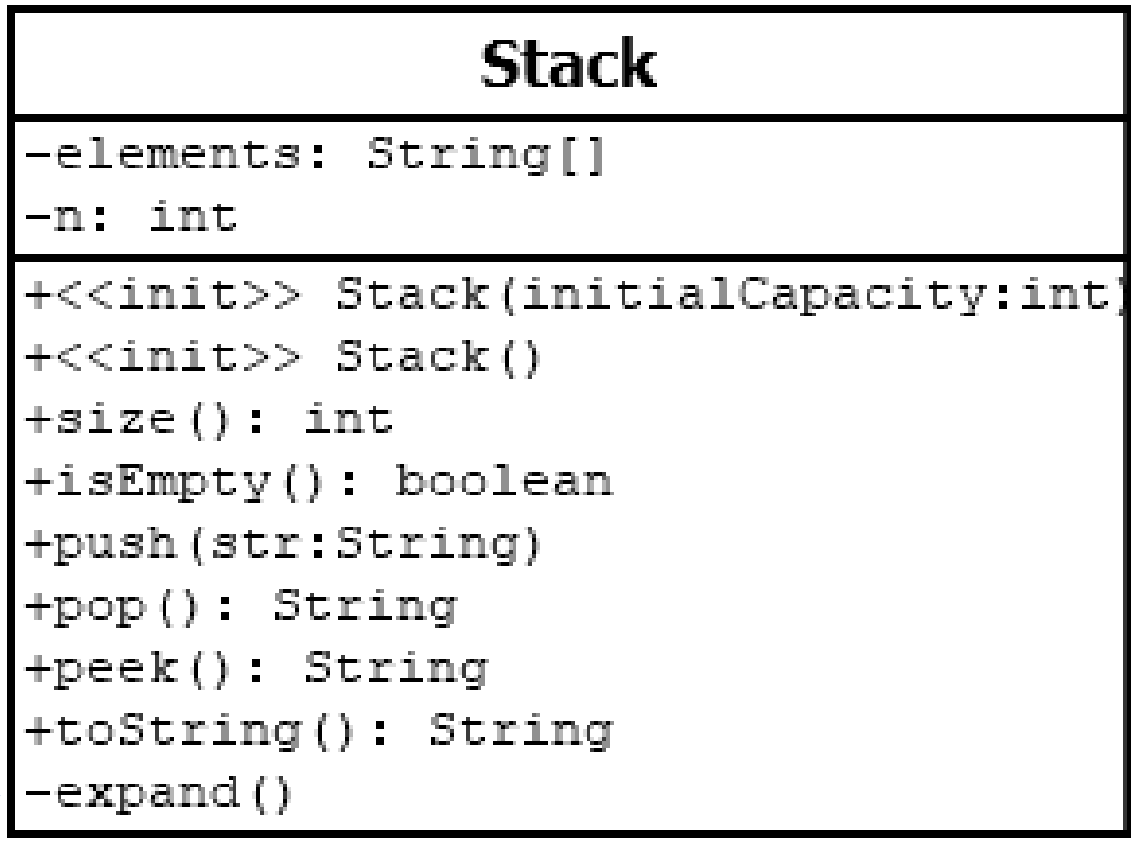
- Datastrukturen Stack används bl.a. till
  - Vända ordningen på en lista med data
  - Anropsstacken för funktioner i ett operativsystem
  - Kompilator: Kontrollera syntax i uttryck
  - …
- Vårt mål är att i ett enkelt fall, en stack för texter, implementera och testa denna datastruktur

Datastrukturen Stack – inre representation

- Internt använder vi en privat array för att lagra elementen …
- … samt en int för att representera aktuellt antal element
  - Skilj på arrayens längd och aktuellt antal element
- Nya element läggs till, push, till höger – top är det sista elementet i arrayen



Datastrukturen Stack, klassdiagram (UML)



Dessutom vill vi att den interna arrayen ”automatiskt” ska utökas när den är full

Stack - implementationen

```
public class Stack {

    private String[] elements;
    private int n; // number of elements; the top of the stack is at n-1

    public Stack(int initialCapacity) {
        if (initialCapacity < 1) throw new IllegalArgumentException("intialCapacity=" + initialCapacity);
        elements = new String[initialCapacity];
        n = 0;
    }

    public Stack() {
        this(100);
    }

    public int size() {
        return n;
    }

    public boolean isEmpty() {
        return n == 0;
    }
```

Stack - implementationen

```java
/**
 * Place a new element at the top of the stack.
 *
 * @param str the new element
 */
public void push(String str) {
    if (n >= elements.length) expand();
    elements[n] = str;
    n++;
}

/**
 * Remove, and return, the element at the top.
 *
 * @return the element at top
 * @throws IllegalStateException if the stack is empty
 */
public String pop() {
    if (isEmpty()) throw new IllegalStateException("empty");
    String element = elements[n - 1];
    elements[n - 1] = null;
    n--;
    return element;
}
```

Stack - implementationen

```
    /**
     * @return the element at the top
     * @throws IllegalStateException if the stack is empty
     */
    public String peek() {
        if (isEmpty()) throw new IllegalStateException("empty");
        return elements[n - 1];
    }

    …

    private void expand() {
        String[] temp = new String[elements.length * 2];
        // copy the references to the new array
        for (int i = 0; i < n; i++) {
            temp[i] = elements[i];
        }
        elements = temp;// the original array will be garbage collected
    }
}
```

Stack – skapa och använda objekt, t.ex. i main

```
Stack stack = new Stack();
System.out.println(stack.toString());

stack.push("A");
stack.push("B");
System.out.println("peek at the top: " + stack.peek()); // B last in, i.e. at the top
System.out.println(stack.toString()); // [B;A]

String str = stack.pop();
System.out.println("popped: " + str); // B
System.out.println(stack.toString()); // [A]

stack.push("C");
stack.push("D");
System.out.println(stack.toString()); // [D,C,A]

// remove all elements
while(!stack.isEmpty()) {
    stack.pop();
    System.out.println(stack.toString());
}
```

Datastrukturen Stack

- Notera att den inre representationen är dold, inkapslad
  - Datamedlemmarna är deklarerade private
- Annan kod/klasser kan endast komma åt objekt av typen stack via det publika gränssnittet, i detta fall de metoder som är deklarerade public
  - push, pop (mutatorer)
  - peek, size, … (access)
- Notera att det är möjligt att ändra den inre implementationen utan att det påverkar annan kod så länge gränssnittet inte ändras
  - I vårt fall kunde vi lägga till en privat metod, expand, utan att det påverkar gränssnittet …
  - … main-metoden påverkas inte av denna ändring av klassens ”inre”